

Problem Set #3 (120 points)

Due date: September 27, 2024, 11:59 PM

REMINDER: As you work on your coding assignments, it is important to remember that passing the examples provided does not guarantee full credit. While these examples can serve as a helpful starting point, it is ultimately your responsibility to create additional tests, ensure that it is functioning correctly and meets all the requirements and specifications outlined in the assignment instructions. Your grade for this assignment is divided in two parts:

50% > Passing all post-due date tests
(no partial credit for failed cases)

50% > Clarity and design of your code
(Class Coding Standards, requirements/specifications)

Before you start...

- You are not allowed to use material that has not been covered in class.
- Your code must be written using Python language.
- Function names must be defined as stated in the function description. Failure to comply with this requirement will result in a 0 score for the visible and post-due date testing.
- No code should be written outside functions, including function calls. All testing code must be done inside the `main` function
- Your functions should NOT have any `input()` calls in its body.
- Your code should be concise and efficient without useless code or redundant code.
- Failure to follow the assignment's requirements will result in a zero score for the assignment even if the autograder gives you full score.
- Code submitted with syntax errors does not receive ANY credit.
- Revise the Grading Notes provided in the Canvas Assignment that includes the evaluation criteria for Clarity and Design.
- You will be writing several functions, but they will all be saved in one file: `PS3.py`. Please save all the functions in this one file or you will lose points.
- Ask questions using our Problem Set 3 channel in Microsoft Teams.

Part 1: The *for* loop

- In this section you **must** use a *for... in range(...)*: style loop

Problem 1.1 (10 pts) Write the Python code to implement the function `primes_count`, that takes two integers a start and a stop, and displays all of the prime numbers within that range (a number is a prime if its number of divisors is 2, 1 and itself). Both the start and the stop parameters are inclusive. After displaying all primes in the range, the function returns the number of primes found. If the start is ever larger than the stop, display the string “ERROR” and return -1.

Examples:

```
>>> primes_count(1,6)
2
3
5
3
>>> primes_count(7, 25)
7
11
13
17
19
23
6
>>> primes_count(8,6)
ERROR
-1
```

Problem 1.2 (10 pts) Radioactive Decay Cobalt-60, a radioactive form of cobalt used in cancer therapy, decays over a period of time. Each year, 12% of the amount present at the beginning of the year will have decayed. Write the Python code to implement the function `decay` that takes two integers, the first one represents the amount of cobalt-60 in a container (in grams) and the second one the numbers of years. The function returns the amount remaining after the given years. Round the amount remaining to two decimal places.  To receive any credit for this problem, you are not allowed to use the Python built-in `round()` function, but you can use the round technique from Problem Set #1.

Examples:

```
>>> decay(10, 5)
5.28
>>> decay(70, 3)
47.7
>>> decay(806, 25)
32.99
```

Problem 1.3 (10 pts) The Fibonacci numbers are a sequence of integers in which the first two elements are 1, and each following element is the sum of the two preceding elements. The mathematical definition of each k^{th} Fibonacci number is the following:

$$F(k) = \begin{cases} F(k-1) + F(k-2), & k > 2 \\ 1, & k \leq 2 \end{cases}$$

For example, the first 12 Fibonacci numbers are 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 and 144. Write the Python code to implement the function `fib_nums` that takes one integer, k , and displays the first k^{th} Fibonacci numbers.

Examples:

```
>>> fib_nums(5)
1
1
2
3
5
>>> fib_nums(8)
1
1
2
3
5
8
13
21
```

Part 2: The *while* loop

- In this section you **must** use a *while* <condition> style loop

Problem 2.1 (10 pts) Write the Python code to implement the function `divisors_count`, that takes a positive integer, `num`, and returns the number of divisors for `num` between 1 to `num`, including `num`.

Examples:

```
>>> divisors_count(89)
2
>>> divisors_count(52)
6
```

Problem 2.2 (10 pts) Write the Python code to implement the function `count_digits` that takes an integer. The function returns the number of digits in that integer. 💡 `%10` and `//10` are useful here.

Examples:

```
>>> count_digits(6589)
4
>>> count_digits(-6589)
4
>>> count_digits(123456788)
9
```

Problem 2.3 (10 pts) Write the Python code to implement the function `divisible_by` that takes two positive integers. The function displays all positive integers less than or equal to the first number that are divisible by the second number from smallest to largest before returning how many numbers were printed.

Examples:

```
>>> out = divisible_by(10, 2)
2
4
6
8
10
>>> out
5
>>> result = divisible_by(3, 1)
1
2
3
>>> result
3
```

Part 3: Pick the best loop

➡ To receive any credit for this problem, you are not allowed to import and use the `math` module! Before you start your implementation first define what kind of loop better satisfies the function requirements. The selection of the proper loop is part of the design grade.

Problem 3.1 (10 pts) An Armstrong number is a number that is equal to the sum of its digits when each digit is raised to the number of digits in the number. Write the Python code to implement the function `is_armstrong_number` that takes a positive integer. The function returns `True` if the input is an Armstrong number, `False` otherwise.

Examples:

```
>>> is_armstrong_number(153)    # 3 digits,  $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$ 
True
>>> is_armstrong_number(169)
False
```

Problem 3.2 (10 pts) Write the Python code to implement the function `factorial_sum` that takes a positive integer. The function returns the sum of factorials from 1 to the given input (inclusive)

Examples:

```
>>> factorial_sum(5)      # 1! + 2! + 3! + 4! + 5!
153
>>> factorial_sum(12)
522956313
```

Problem 3.3 (10 pts) A perfect number is an integer whose sum of integer divisors (excluding the number itself) add up to the number. Following are the first four perfect numbers:

$$6 = 1 + 2 + 3$$

$$28 = 1 + 2 + 4 + 7 + 14$$

$$496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248$$

$$8128 = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 127 + 254 + 508 + 1016 + 2032 + 4064$$

Write the Python code to implement the function `is_perfect_number` that takes a positive integer. The function returns True if the number is a perfect number, False otherwise. 💡 Use the same approach as the `is_prime` hands-on lecture to avoid checking all numbers up to the given input.

Problem 3.4 (10 pts) Write the Python code to implement the function `temp_sum`. Your function should take two integer arguments, start and end, that represent the range of degrees Celsius. Your function returns the sum of the equivalent Fahrenheit temperature for the Celsius temperatures from start to end (inclusive) in increments of 5 degrees. As a reminder $^{\circ}\text{F} = (^{\circ}\text{C} * 9/5) + 32$

Examples:

```
>>> temp_sum(10,30)
340.0
>>> temp_sum(50,3000)
1641207.0
>>> temp_sum(12,73)
1398.8000000000002
```

Problem 3.5 (10 pts) The world population reached 7 billion people on October 21, 2011, and was growing at the rate of 1.1% each year. Assuming that the population continues to grow at the same rate, define a function named `estimate`, that takes one argument to represent the number of billions and returns the numbers of years when will the population reach the given billions.

Examples:

```
>>> estimate(8)
2024
>>> estimate(10)
2044
>>> estimate(11.5)
2057
```

Problem 3.6 (10 pts) The proper factors of a number are all of the factors other than the number itself. For instance, the factors of 10 are 1, 2, 5, and 10, but the proper factors of 10 are only 1, 2, and 5. Every number can be classified as abundant, deficient, or perfect, according to the following definitions:

- Abundant: The sum of the proper factors is greater than the number itself.
- Deficient: The sum of the proper factors is less than the number itself.
- Perfect: The sum of the proper factors is equal to than the number itself.

Define a function named `classify`. Your function should take one argument and display a string to classify each number in the inclusive range 2 to the given input as abundant, deficient or perfect using the format "*value* is ____". Your string must match the given examples' format, no partial credit for output that does not match.

Examples:

```
>>> classify(25)
2 is deficient
3 is deficient
4 is deficient
5 is deficient
6 is perfect
7 is deficient
8 is deficient
9 is deficient
10 is deficient
11 is deficient
12 is abundant
13 is deficient
14 is deficient
15 is deficient
16 is deficient
17 is deficient
18 is abundant
19 is deficient
20 is abundant
21 is deficient
22 is deficient
23 is deficient
24 is abundant
25 is deficient
```